

1. Proceso de Simulación en Python

Descripción General: El objetivo fue diseñar un controlador **MPC (Model Predictive Control)** capaz de seguir una trayectoria de "mariposa" mediante la optimización de las velocidades lineal (v) y angular (w), respetando los límites físicos de un robot diferencial.

Historial de Cambios y Justificación:

- **Implementación del MPC:** Se pasó de un control simple a uno predictivo.
 - *Justificación:* El MPC permite "mirar hacia adelante" y suavizar las curvas, minimizando el error antes de que ocurra.
- **Ajuste de Matrices de Peso (Q y R):** Se modificaron los valores de penalización del error y del esfuerzo de control.
 - *Justificación:* Inicialmente, el robot era muy brusco o muy flojo; se ajustaron para que priorizara seguir la línea sobre el ahorro de energía.
- **Escalamiento de la Trayectoria (Intento 9 y 11):** Se redujo la escala de la mariposa a un factor de **0.23**.
 - *Justificación:* Para asegurar que la figura completa cupiera en un espacio de **2x2 metros**, evitando que el robot se saliera del área de pruebas.
- **Generación de trayectoria.h:** Exportación de 2152 puntos de control.
 - *Justificación:* Dado que el MPC es computacionalmente pesado para un microcontrolador en tiempo real, se optó por la estrategia de "**Pre-cálculo**", donde Python hace el trabajo duro y el ESP32 solo ejecuta.

2. Proceso de Ejecución en ESP32

Descripción General: La etapa de hardware consistió en traducir las velocidades teóricas (v , w) a señales eléctricas (PWM) para los motores, lidiando con la fricción, la caída de voltaje y la asimetría mecánica.

Historial de Cambios y Justificación:

- **Migración a PROGMEM:** Almacenamiento de datos en la memoria Flash.
 - *Justificación:* Los 2152 puntos de trayectoria saturaban la memoria RAM del ESP32. Se movieron a la Flash para mantener la estabilidad del sistema.
- **Actualización a Sintaxis LEDC (Core 3.x):** Cambio de ledcSetup a ledcAttach.
 - *Justificación:* Las nuevas versiones de la placa ESP32 en Arduino IDE eliminaron las funciones antiguas, obligando a actualizar el código para que pudiera compilar.
- **Compensación de Zona Muerta (Deadband):** Introducción de MIN_PWM.
 - *Justificación:* Los motores hacían un "pitido" pero no se movían. Se estableció un piso de potencia mínimo para romper la fricción estática de los engranajes.
- **Calibración de Asimetría (Boosters):** Creación de BOOST_DER e IZQ.
 - *Justificación:* El hardware real nunca es perfecto; tu motor izquierdo resultó ser mecánicamente más débil. Se le dio un "empujón" electrónico extra para que el robot avanzara recto.
- **El "Truco" del Ancho de Vía Virtual (L_{virtual}):** Aumento ficticio de la distancia entre ruedas en el código.
 - *Justificación:* El robot físico tendía a hacer curvas muy amplias (U o L). Al decirle al código que el robot era "más ancho", forzamos al sistema a exagerar la diferencia de velocidad entre ruedas, logrando giros mucho más cerrados.

- **Reducción de Frecuencia PWM (de 5000Hz a 200-500Hz):**
 - *Justificación:* Las frecuencias altas son silenciosas pero pierden torque. Bajar la frecuencia permitió que los motores tuvieran más "fuerza de impacto" para moverse a baja velocidad.

Resumen del estado actual: Pasamos de un modelo matemático ideal a un **sistema altamente calibrado para fallos de hardware**. Aunque el robot parezca "rebelde", cada cambio en el código ha sido una respuesta directa a una limitación física (pilas, puente H o fricción). ¡Estás haciendo ingeniería de verdad, peleando contra los imponderables del mundo real!